



ambient
scientific

Ambient Scientific Inc.

CNN Training Tool-Chain
Framework

Version: 1

Date: Nov/15/2024

Confidentiality Statement

This document contains highly confidential and proprietary information and data of Ambient Scientific and its affiliates. This document is provided, on the express condition that such information and data will not be used, disclosed, or reproduced in any form, in whole or in part, for any purpose (other than solely for evaluation purposes by authorized representatives with a need to know), without the express written approval of Ambient Scientific. The right to use, disclose, and reproduce such information and data shall be governed by the service agreement between the client and Ambient Scientific.

Warnings and Disclaimer

The information provided is on an as is basis. Every effort has been made to make this book as complete and as accurate as possible, but no warranty or fitness is implied. The authors and the publisher shall have neither liability nor responsibility to any person or entity with respect to any loss or damages arising from the information contained in this book. Screenshots and data used within are for illustration purpose and may vary from the actual application.

Ambient Scientific – All rights reserved.

Document Change History Log

Version No.	Date	Author / Editor	Details of Change
1.0	15/11/2024	Praveen	1 st version

Document Acceptance

Name	Function	Date

Highly Confidential

Table of Contents

1	Project Definition.....	6
1.1	Scope.....	6
2	Applications Supported using CNN	7
3	Work-Flow.....	7
3.1	Step 1 - Import TF/ Keras Libraries:.....	8
3.2	Step 2 - Set Path to Load/Import Data	9
3.3	Step 3 - Generate Data for Model Training:	10
3.4	Step 4 - Feature Extraction:	11
3.5	Step 5 - Load Data and reshape for Model Training.....	12
3.6	Step 6 – Split Data into Train/Test:.....	13
3.7	Step 7 – AI Algorithm for Model Training	14
3.8	Step 8 – Save/Load the Model.....	16
3.9	Step 9 – Test the model in Python:.....	17
3.10	Step 10 – Quantize the Model to INT8 and save in TFLite:.....	18
3.11	Step 11 – Test the TFLite Model:	19

1 Project Definition

1.1 Scope

A Training Tool chain which shows step by step training procedure from data Collection to Model Generation & Testing.

This document should be used in tandem with model training python (.py) files for best use of tool chain. Python files are provided in sample applications.

2 Applications Supported using CNN

Speech: Single Phrase Command Words.

Vision: Driver Monitoring System, Face ID.

Sensor Fusion: Fall Detection

3 Work-Flow

- Step 1 - Import TF/ Keras Libraries
- Step 2 - Set Path to Load/Import Data
- Step 3 – Generate Data for Model Training
- Step 4 – Feature Extraction
- Step 5 – Load Data and reshape for Model Training
- Step 6 – Split Data into Train/Test
- Step 7 – AI Algorithm for Model Training
- Step 8 – Save/Load the Model
- Step 9 – Test the model in Python.
- Step 10 – Quantize the Model to INT8 and save in TFLite.
- Step 11 – Test the TFLite Model

Note: Don't use anaconda prompt for any installations.

Uninstall any pre-existing packages or python versions before starting with installation.

3.1 Step 1 - Import TF/ Keras Libraries:

Make sure `python 3.10.11` is installed in the PC.

All the packages with the versions are mentioned in requirements.txt

Use following command for installation using command prompt.

```
pip install -r requirements.txt
```

All the packages will be installed automatically using above command.

Note: Instructions are provided in Sample application (Python code) for each use case.

3.2 Step 2 - Set Path to Load/Import Data

Import/Load data from the native system to python IDE (.py file).

Specify the data path where the Audio/Image/Sensor sample data is present.

Name it with a user-defined variable.

Variable = “Path of Training Data”

E.g., DATA_PATH = ‘Dataset.’

3.3 Step 3 - Generate Data for Model Training:

Based on the Application, the Data Generation will vary.

Vision application:

Image augmentation is a technique commonly used in computer vision and deep learning tasks, especially in the context of training convolutional neural networks (CNNs) for tasks like image classification, object detection, and segmentation. The goal of image augmentation is to artificially increase the size of the training dataset by applying a variety of transformations to the existing images.

These transformations include:

- **Rotation:** Rotating the image by a certain angle.
- **Translation:** Shifting the image horizontally or vertically.
- **Scaling:** Zooming in or out of the image.
- **Shearing:** Tilting the image along its axes.
- **Flipping:** Mirroring the image horizontally or vertically.
- **Brightness adjustment:** Changing the brightness of the image.
- **Contrast adjustment:** Changing the contrast of the image.
- **Noise addition:** Adding random noise to the image.
- **Color channel manipulation:** Altering the intensity of color channels.

Refer to **Image_Augmentation.py** for the above operations performed through code.

Similarly, for Speech and Sensor Data, we are going to generate data using DVK (explained in application specific code – Please Refer to corresponding folder.)

Note: Instructions are provided in Sample application (Python code) for each use case.

3.4 Step 4 - Feature Extraction:

Speech Application: (Custom Feature Extraction)

Frame the Signal: The audio signal is divided into short frames of 32 milliseconds (256 samples at 8KHz). The frames are usually windowed using functions like Hamming or Hanning to reduce spectral leakage.

Fast Fourier Transform (FFT): For each frame, the Fourier Transform is applied to convert the signal from the time domain to the frequency domain. This provides the spectrum of the signal.

Mel Filtering: The spectrum is then passed through a bank of Mel filters, which are triangular-shaped bandpass filters spaced uniformly on the mel scale. The mel scale is a perceptual scale of pitches that approximates the human auditory system's response to different frequencies.

Logarithm: After filtering, the logarithm of the filter bank outputs is taken. This helps to mimic the logarithmic perception of loudness by the human auditory system.

Refer to **Feature_Extraction.py** for the complete implementation of **Custom Feature Extraction** through code.

Vision Application:

Feature extraction is taken care during convolution processing for images/Sensor Data. So, no special feature suggested.

Sensor Application:

Like vision application, Feature extraction is taken care during convolution processing for images/Sensor Data. So, no special feature suggested.

Note: User can add their own feature processing if needed.

Common feature extraction processes used for sensor applications are Mean (), Std (), Mad(), Energy(),....

Note: Instructions are provided in Sample application (Python code) for each use case.

3.5 Step 5 - Load Data and reshape for Model Training.

In this step, user have to load Training data (common in all applications) and can see the data shape/Dimensions by printing its shape.

Based on the application specifications, User have to reshape the size of the input Data, which is going to be used for model training.

Note: Instructions are provided in Sample application (Python code) for each use case.

3.6 Step 6 – Split Data into Train/Test:

The complete Data will be split into Train & Test Data (80:20)

80% of Total Data will be used for Model Training (seen Data)

Remaining 20% will be considered as unseen data and will be used for Testing.

Note: Instructions are provided in Sample application (Python code) for each use case.

3.7 Step 7 – AI Algorithm for Model Training

Defining the Algorithm for model Training involves specifying its structure, including the types and arrangements of layers or components, along with any parameters or hyperparameters. The model architecture defines how data flows through the model and how it is transformed at each step. Here's how you can define the architecture of a model:

Choose Model Type: Determine the type of model you'll be using based on the nature of your data and the task at hand. For example, if you're working with image data, you might choose a Convolutional Neural Network (CNN)

Define Input and Output Layers: Specify the input shape of your data, which depends on the dimensionality and structure of your input data. Likewise, define the output layer based on the type of task you're solving (e.g., classification, regression). For classification tasks, the output layer might consist of a SoftMax layer for multi-class classification or a sigmoid layer for binary classification.

Choose Hidden Layers and Activation Functions: Decide on the number of hidden layers and the number of neurons in each layer. Choose appropriate activation functions for each layer to introduce non-linearity into the model. Common activation functions include ReLU (Rectified Linear Unit), tanh (Hyperbolic Tangent), and sigmoid.

Compiler Supported Operators

Before finalizing the Neural Network operators for training, one has to go through the list of Operators supported by GPX Processor compiler to compile the model in GPX.

SN	TF/Keras NN Operators	Sub-Operators
1	Dense	
2	Flatten	
3	Dropout	
4	Conv2D	Symmetric Input
5		Symmetric Kernel
6		Symmetric Stride
7		Asymmetric Input
8		Asymmetric Kernel
9		Padding
10	MaxPool2D	Symmetric Input
11		Symmetric Kernel
12		Symmetric Stride
13		Asymmetric Input
14		Asymmetric Kernel
15	data_format	channels_first (NCHW)
16	Activations	Sigmoid
17		TanH
18		Relu
19		Softmax

Sample Model:

```
from keras import layers, models
model = models.Sequential()
model.add(layers.Conv2D(10, (5, 5), strides=3, activation='relu', input_shape=(1,162, 162)))
model.add(layers.Dropout(0.3))
model.add(layers.MaxPooling2D((2, 2), strides= 2))
model.add(layers.Conv2D(20, (5,5), strides=2, activation='relu'))
model.add(layers.Dropout(0.3))
model.add(layers.MaxPooling2D((2, 2), strides= 2))
model.add(layers.Flatten())
model.add(layers.Dense(20, activation='relu'))
model.add(layers.Dropout(0.3))
model.add(layers.Dense(5, activation='softmax'))
model.summary()
```

Compile the Model:

Specify Regularization and Optimization:

Include regularization techniques such as dropout, L1/L2 regularization, or batch normalization to prevent overfitting and improve generalization. Choose an optimization algorithm (e.g., SGD, Adam, RMSprop) and specify its parameters (e.g., learning rate, momentum).

Select Loss Function:

Choose a loss function appropriate for your task, such as categorical cross-entropy for classification or mean squared error for regression. The choice of loss function depends on the output type and the problem you're solving.

```
model.compile(optimizer='Adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
```

Fit the Model:

To fit the defined model to your data, you'll need to prepare your data and then train the model using the fit ()

```
history = model.fit(train_images, train_labels, epochs=50, batch_size = 256,
                    validation_data=(test_images, test_labels))
```

Note: Instructions are provided in Sample application (Python code) for each use case.

3.8 Step 8 – Save/Load the Model

Save the model with a user-defined name with an extension of (.h5) as shown below

```
model.save("audio_model.h5")
```

```
model.save("vision_model.h5")
```

```
model.save("sensor_model.h5")
```

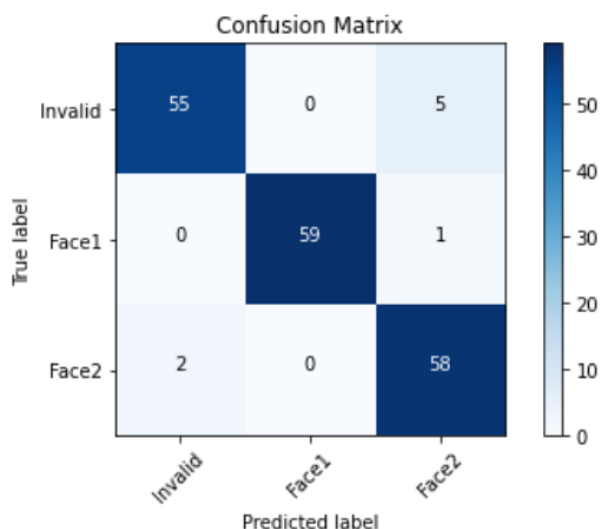
Note: Instructions are provided in Sample application (Python code) for each use case.

3.9 Step 9 – Test the model in Python:

Testing a model in Python involves evaluating its performance on a separate dataset, typically called a test set, which was not used during the training phase (Unseen Data).

Evaluating Performance:

With the model predictions and the ground truth labels (or values) from the test data, User can evaluate the model's performance using appropriate metrics. Common metrics for classification tasks include accuracy, precision, recall, F1-score (Confusion Matrix).



Note: Instructions are provided in Sample application (Python code) for each use case.

3.10 Step 10 – Quantize the Model to INT8 and save in TFLite:

Quantization is a process used to reduce the precision of numerical values in a model. Converting a model to INT8 (8-bit integer) quantization is a common technique to make the model more memory and computationally efficient, especially for deployment on resource-constrained devices.

TensorFlow Lite is a framework for deploying machine learning models on mobile and edge devices. TFLite provides tools and utilities for converting TensorFlow models into a format optimized for deployment, including quantization to INT8. Once a model is quantized to INT8, it can be converted to the TFLite format for efficient execution on devices.

To quantize a TensorFlow model to INT8 and save it in TFLite format, you typically use the TensorFlow Lite Converter. This involves loading the trained TensorFlow model, applying quantization to the desired parts of the model (e.g., weights, activations), and then converting the model to the TFLite format. The converter automatically handles the computation of scaling factors and other necessary transformations during the quantization process.

Note: Instructions are provided in Sample application (Python code) for each use case.

3.11 Step 11 – Test the TFLite Model:

Testing a TensorFlow Lite (TFLite) model involves evaluating its performance on a dataset to ensure that it produces accurate predictions or outputs. Input has to be in float32 format to start tflite conversion.

Representative dataset has to be $\leq$ number of samples available.

Convert the Keras model to TFLite format with int8 quantization

Define function to evaluate the TFLite model

Evaluate the TFLite model on the test data

Generate confusion matrix.

The above-mentioned steps are explained in-detail in the specific application (Please refer to specific application project)

Note: Instructions are provided in Sample application (Python code) for each use case.